

3. Assignment การสร้าง diagrams จาก mermaid

สร้าง Mermaid Diagram สำหรับระบบ AI Monitoring ของ Server

สถาปัตยกรรมระบบ AI-Powered Server Monitoring System ถูกแบ่งออกเป็น 5 เลเยอร์หลักตามมาตรฐาน Data Pipeline (Collection → Ingestion → Storage → AI Inference → Action/Dashboard):

```
mermaid
graph TD
    subgraph L1 ["1. Data Sources (Server Infrastructure)"]
        S_BareMetal["Bare-Metal / Virtual Machines"]
        S_K8s["Containers / Kubernetes"]
        S_App["Application & Network Services"]
        A_Metric["Node / Telegraf Exporter (CPU, RAM, Disk, I/O)"]
        A_Log["Fluentbit / Vector (System & App Logs)"]
        A_Trace["OpenTelemetry Agent (APM Traces)"]
        S_BareMetal --> A_Metric
        S_K8s --> A_Log
        S_App --> A_Trace
    end
    subgraph L2 ["2. Data Ingestion & Streaming"]
        Kafka["Apache Kafka / RabbitMQ (Message Broker & Buffer)"]
        A_Metric --> Kafka
        A_Log --> Kafka
        A_Trace --> Kafka
    end
    subgraph L3 ["3. Storage Layer"]
        TSDB["Time-Series DB (Prometheus / VictoriaMetrics)"]
        LogDB["Log / Search DB (Elasticsearch / OpenSearch)"]
        Kafka --> TSDB
        Kafka --> LogDB
    end
    subgraph L4 ["4. AI & Analytics Engine (AI/ops)"]
        direction TB
        Feature["Feature Processing & Window Aggregator"]
        subgraph Models ["AI Core Models"]
            M_Anomaly["Anomaly Detection (Isolation Forest / Autoencoders)"]
            M_Pred["Capacity & Load Forecasting (Prophet / LSTM)"]
            M_RCA["Root Cause Analysis (Graph Neural Network / LLM Agents)"]
        end
        TSDB --> Feature
        LogDB --> Feature
        Feature --> M_Anomaly
        Feature --> M_Pred
        Feature --> M_RCA
    end
    subgraph L5 ["5. Action, Alerting & Visualization"]
        Dash["Unified Dashboard (Grafana / Kibana)"]
        AlertRouter["Decision Engine & Threshold Router"]
        Act_SelfHeal["Automated Remediation (Ansible / K8s Auto-heal)"]
        Act_Notify["Alert Notification (Slack, Discord, PagerDuty, OpsGenie)"]
        M_Anomaly --> Dash
        M_Pred --> Dash
        M_Anomaly --> AlertRouter
        M_RCA --> AlertRouter
        AlertRouter -- "Auto-Resolvable Issue" --> Act_SelfHeal
        AlertRouter -- "Critical Incident / Manual Fix" --> Act_Notify
    end
    classDef source fill:#E3F2FD,stroke:#1565C0,stroke-width:2px;
    classDef ingestion fill:#FFF3E0,stroke:#E65100,stroke-width:2px;
    classDef storage fill:#E8F5E9,stroke:#2E7D32,stroke-width:2px;
    classDef ai fill:#F3E5F5,stroke:#A1887F,stroke-width:2px;
    classDef action fill:#FEEBEE,stroke:#C0392B,stroke-width:2px;
    class S_BareMetal,S_K8s,S_App,A_Metric,A_Log,A_Trace source;
    class Kafka ingestion;
    class TSDB,LogDB storage;
    class Feature,M_Anomaly,M_Pred,M_RCA ai;
    class Dash,AlertRouter,Act_SelfHeal,Act_Notify action;
```

ขอความช่วยเหลือจาก Gemini

Flash

end

subgraph L4 ["4. AI & Analytics Engine (AI/ops)"]

direction TB

Feature["Feature Processing & Window Aggregator"]

subgraph Models ["AI Core Models"]

M_Anomaly["Anomaly Detection (Isolation Forest / Autoencoders)"]

M_Pred["Capacity & Load Forecasting (Prophet / LSTM)"]

M_RCA["Root Cause Analysis (Graph Neural Network / LLM Agents)"]

end

TSDB --> Feature

LogDB --> Feature

Feature --> M_Anomaly

Feature --> M_Pred

Feature --> M_RCA

end

subgraph L5 ["5. Action, Alerting & Visualization"]

Dash["Unified Dashboard (Grafana / Kibana)"]

AlertRouter["Decision Engine & Threshold Router"]

Act_SelfHeal["Automated Remediation (Ansible / K8s Auto-heal)"]

Act_Notify["Alert Notification (Slack, Discord, PagerDuty, OpsGenie)"]

M_Anomaly --> Dash

M_Pred --> Dash

M_Anomaly --> AlertRouter

M_RCA --> AlertRouter

AlertRouter -- "Auto-Resolvable Issue" --> Act_SelfHeal

AlertRouter -- "Critical Incident / Manual Fix" --> Act_Notify

end

classDef source fill:#E3F2FD,stroke:#1565C0,stroke-width:2px;

classDef ingestion fill:#FFF3E0,stroke:#E65100,stroke-width:2px;

classDef storage fill:#E8F5E9,stroke:#2E7D32,stroke-width:2px;

classDef ai fill:#F3E5F5,stroke:#A1887F,stroke-width:2px;

classDef action fill:#FEEBEE,stroke:#C0392B,stroke-width:2px;

class S_BareMetal,S_K8s,S_App,A_Metric,A_Log,A_Trace source;

class Kafka ingestion;

class TSDB,LogDB storage;

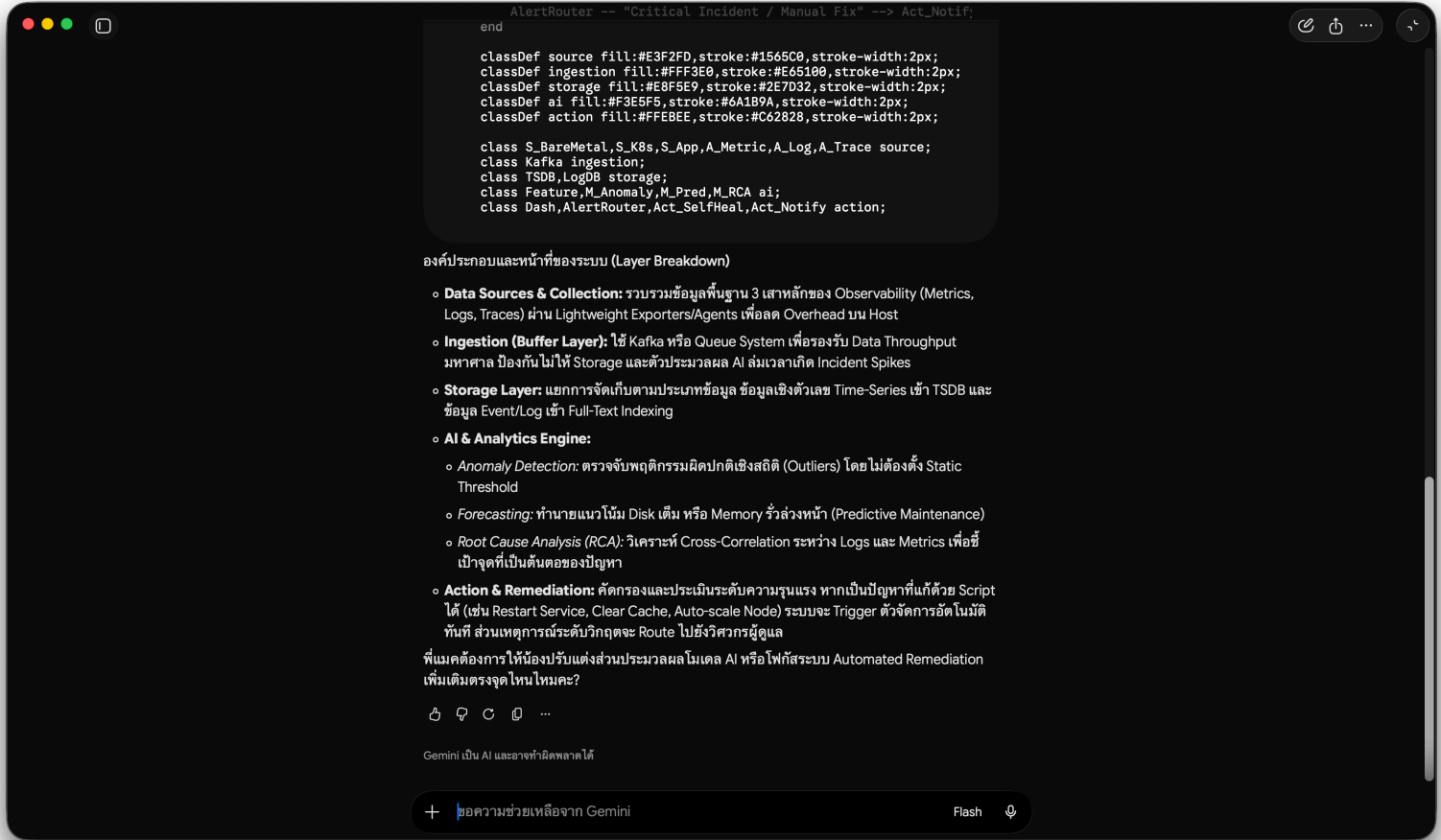
class Feature,M_Anomaly,M_Pred,M_RCA ai;

class Dash,AlertRouter,Act_SelfHeal,Act_Notify action;

องค์ประกอบและหน้าที่ของระบบ (Layer Breakdown)

ขอความช่วยเหลือจาก Gemini

Flash



องค์ประกอบและหน้าที่ของระบบ (Layer Breakdown)

- **Data Sources & Collection:** รวบรวมข้อมูลพื้นฐาน 3 เสาหลักของ Observability (Metrics, Logs, Traces) ผ่าน Lightweight Exporters/Agents เพื่อลด Overhead บน Host
- **Ingestion (Buffer Layer):** ใช้ Kafka หรือ Queue System เพื่อรองรับ Data Throughput มหาศาล ป้องกันไม่ให้ Storage และตัวประมวลผล AI ล่มเวลาเกิด Incident Spikes
- **Storage Layer:** แยกการจัดเก็บตามประเภทข้อมูล ข้อมูลเชิงตัวเลข Time-Series เข้า TSDB และ ข้อมูล Event/Log เข้า Full-Text Indexing
- **AI & Analytics Engine:**
 - *Anomaly Detection:* ตรวจจับพฤติกรรมผิดปกติเชิงสถิติ (Outliers) โดยไม่ต้องตั้ง Static Threshold
 - *Forecasting:* ทำนายแนวโน้ม Disk เต็ม หรือ Memory รั่วล่วงหน้า (Predictive Maintenance)
 - *Root Cause Analysis (RCA):* วิเคราะห์ Cross-Correlation ระหว่าง Logs และ Metrics เพื่อชี้เป้าจุดที่เป็นต้นตอของปัญหา
- **Action & Remediation:** คัดกรองและประเมินระดับความรุนแรง หากเป็นปัญหาที่แก้ด้วย Script ได้ (เช่น Restart Service, Clear Cache, Auto-scale Node) ระบบจะ Trigger ตัวจัดการอัตโนมัติทันที ส่วนเหตุการณ์ระดับวิกฤตจะ Route ไปยังวิศวกรผู้ดูแล

ทีมต้องการให้น้องปรับแต่งส่วนประมวลผลโมเดล AI หรือโฟกัสระบบ Automated Remediation เพิ่มเติมตรงจุดไหนไหมคะ?

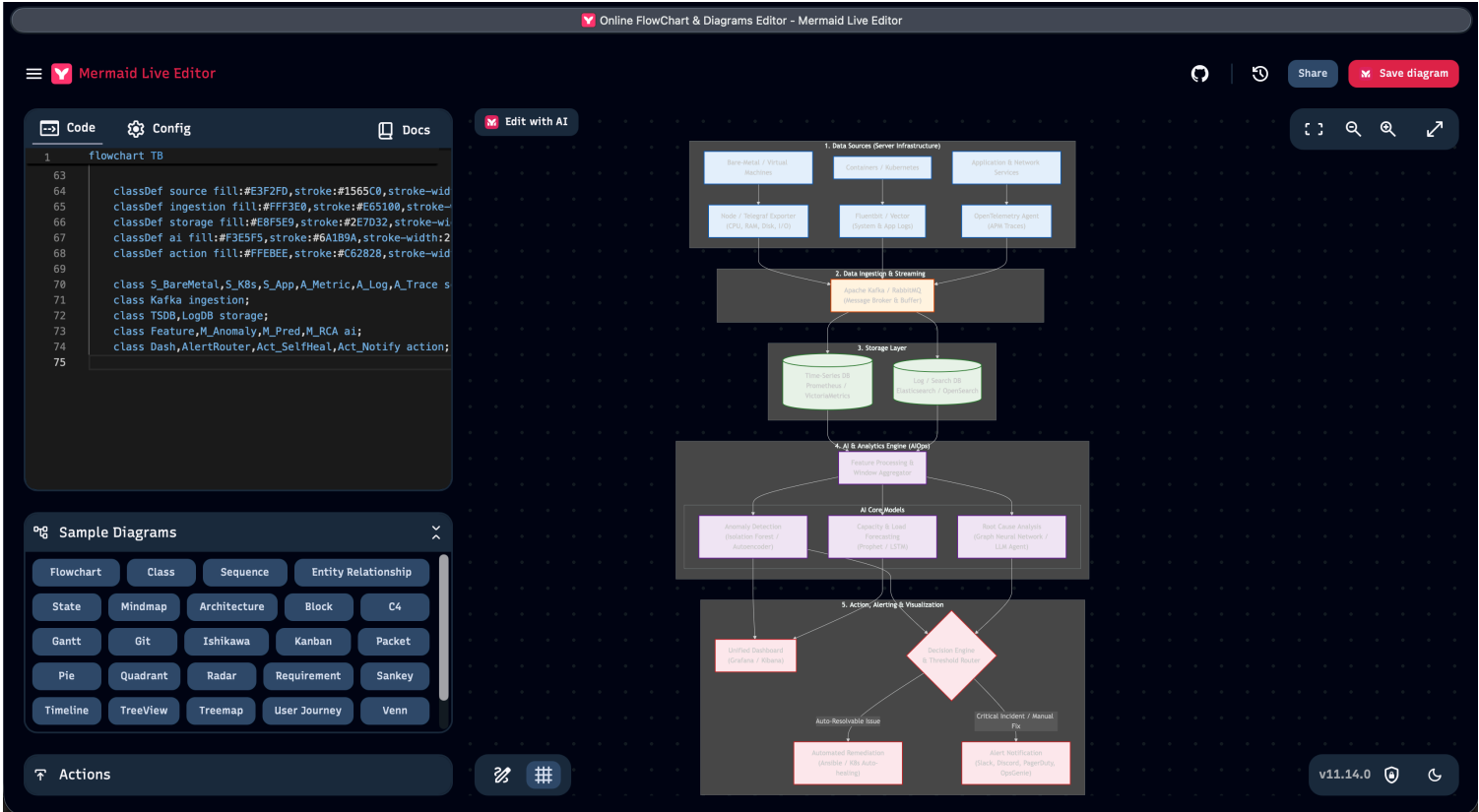
🗑️ 🗑️ 🗑️ 🗑️ ...

Gemini เป็น AI และอาจทำผิดพลาดได้

+ ขอความช่วยเหลือจาก Gemini

Flash

Output



Full file Output

